Predicting Track Popularity on Last.fm with Linear Regression

Dana Belgibek
SDU University, SDS307 Modern Regression Analysis

October 2026

Abstract

We study which factors are associated with the popularity of music tracks, measured by the number of plays (`playcount`) on Last.fm. We collected 12,035 tracks by 2,514 artists through the public Last.fm API and built 23 features of the title, the track metadata and the artist. Playcount is strongly right-skewed, so we model its logarithm and split the data by artist. Linear regression explains 71% of the variance for unseen artists (test $R^2 = 0.71$); diagnostics reveal heteroscedasticity and multicollinearity.

1 Introduction and Overview

Motivation. Record labels and streaming platforms must decide which tracks to promote or recommend first, often when a track has little listening history. A model linking public metadata (artist reach, available descriptions and tags, title properties) to the number of plays can support such decisions and shows which factors matter.

Problem and data. We predict the number of plays of a track from 23 features. The data come from the public Last.fm API (collected on October 7, 2026): 12,035 tracks by 2,514 artists from 14 genre tags.

Target. `playcount` is the number of plays of a track (a non-negative count treated as continuous), available for all tracks. Its median is 1,426,212 and its maximum 123,075,880. The distribution is strongly right-skewed (skewness 4.22); $\log(1 + \texttt{playcount})$ has skewness -0.33 and ranges from 3.7 to 18.6 (Figure 1), so the models use the log target.

2 Theoretical Background

Multiple linear regression models $y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i$; ordinary least squares (OLS) minimises $\sum_i (y_i - \hat{y}_i)^2$. Inference assumes linearity, independent errors, constant error variance and normal errors. Models are compared with

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad R^2_{adj} = 1 - (1 - R^2)\frac{n-1}{n-p-1}, \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}, \quad \text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|. \tag{1}$$

Multicollinearity is measured by $\text{VIF}_j = 1/(1 - R_j^2)$, where R_j^2 comes from regressing feature j on the other features; constant variance is checked with residual plots and the Breusch–Pagan test, normality with a Q-Q plot. A categorical feature with k categories gets $k - 1$ dummy variables; the omitted one is the reference category (this avoids the dummy variable trap).

3 Algorithm Implementation and Development

3.1 Data collection

Our first attempt used the Spotify Web API, but its audio-features endpoint returned HTTP 403 for our application, so we switched to Last.fm. We used three official API methods: `tag.getTopTracks` (candidates, up to 20 pages of 50 tracks per genre tag), `track.getInfo` (plays, duration, album, tags,

description) and `artist.getInfo` (listeners, plays, biography). Collection stopped at about 12,000 candidates (12,037) from 14 tags: indie, alternative, electronic, hip-hop, rock, jazz, metal, folk, punk, soul, r&b, reggae, country and blues. Each request was followed by a 1-second pause, failed requests were retried, and results were saved after every request so that the collection could be resumed (`scraper.py`).

Ethics. Only documented API methods were used, with a registered key and an identifiable User-Agent, one request per second; no website pages were scraped. On October 11, 2026 we checked `robots.txt` of `www.last.fm` (it disallows only website paths such as `/search`, `/forum`, `/label`; the file of the API host could not be retrieved) and read the API Terms of Service [5]. The data (about 2 MB, below the 100 MB cap) are used only for this non-commercial course project. The Terms ask academic users to contact Last.fm beforehand, which we did not do. No data about Last.fm users were collected, and in the shared dataset names are replaced by numeric identifiers.

3.2 Cleaning

Of 12,037 candidates, 12,035 tracks remained after merging and dropping records without an API response; no duplicates or non-positive play counts were found. Of the 32 raw features, nine were removed (`cleaning.py`): four title flags that are almost always zero (`has_remix`, `has_live`, `has_edit`, `has_version`), four constant or near-constant features (`n_similar_artists`, `n_artist_tags`, `album_position`, `has_artist_bio`) and `top_tag` (80.8% "none"). The API apparently returns a fixed number of similar artists and tags, so these features carry no information, which is an unsuccessful part of the collection. The 321 missing values (2.7%) of `duration_min` are filled with the training median after the split.

3.3 Features and limitations

Table 1: The 23 features after cleaning (identifiers and target not counted).

Group	No.	Content
Track title	7	length, word count, flags for featuring, parentheses, digits, non-ASCII, title equal to album
Artist name	3	length, word count, collaboration flag
Track metadata	5	duration, has album, `n_track_tags`, `has_wiki`, `wiki_length`
Artist statistics	6	listeners, plays, `log_artist_listeners`, plays per listener, biography length, on tour
Sampling	2	`genre_tag` (categorical), `rank_in_genre`

Limitations: (i) `rank_in_genre` takes only the values 1–50 and repeats on every page, so it is the position on a page of the tag list, not the rank in the genre; (ii) `n_track_tags` is 0 for 9,729 tracks and 5 for 2,305 (one track has 3), so it only indicates whether tags exist; (iii) tracks come from top lists of tags, so the sample is biased towards popular tracks and genre proportions follow from the collection (blues has only 255 tracks); (iv) the 2,514 artists have about 4.8 tracks each, so observations are not independent; (v) `artist_playcount` includes the plays of the track itself (possible leakage).

3.4 Split and model fitting

The data are split 80/20 by artist (`GroupShuffleSplit`, seed 42): 9,804 tracks of 2,011 artists for training and 2,231 tracks of 503 artists for testing. The target is $\log(1 + \texttt{playcount})$. Because `artist_playcount` and `wiki_length` are strongly skewed, we use their logarithms and drop the raw `artist_listeners` (its logarithm is kept). `genre_tag` is encoded with dummy variables (reference category *alternative*), giving 34 predictors. We fit OLS with `statsmodels` and `scikit-learn`; Ridge and Lasso (standardised features) serve as extensions. The code is in `analysis.ipynb`.

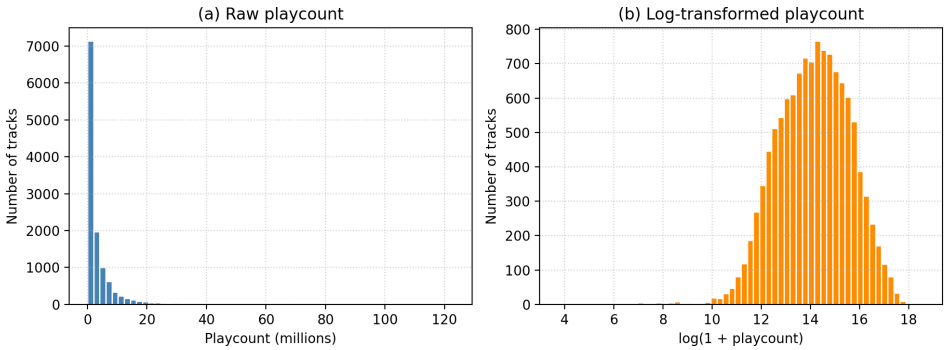


Figure 1: Distribution of `playcount` before (a) and after (b) the log transformation.

4 Computational Results

4.1 Exploratory analysis

The strongest correlations with the log target (Figure 2) are `log_artist_listeners` (0.64), `n_track_tags` (0.62), `wiki_length` (0.57), `artist_listeners` (0.56) and `has_wiki` (0.55). Title and artist-name features are weakly correlated (for example `artist_length` −0.12), and `rank_in_genre` is almost uncorrelated (−0.025).

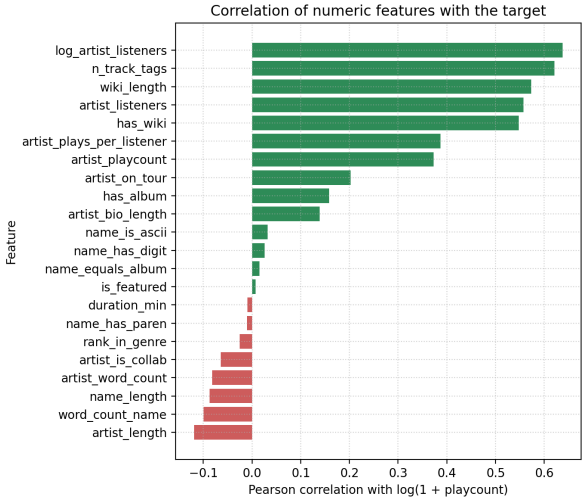


Figure 2: Pearson correlation of the numeric features with $\log(1 + \texttt{playcount})$.

4.2 Models

Table 2: Test performance on the log target (2,231 tracks by 503 unseen artists).

Model	R^2 train	R^2 test	Adj. R^2 test	RMSE	MAE
Baseline (training mean)	0.000	−0.006	−0.022	1.473	1.199
OLS	0.742	0.714	0.709	0.786	0.611

OLS explains 71% of the variance for unseen artists; the RMSE of 0.79 means a typical error by a factor of $e^{0.79} \approx 2.2$. The small gap between training and test R^2 shows no strong overfitting, and Ridge and Lasso do not help (test R^2 0.714 and 0.710). The largest effects belong to metadata availability and artist reach: tracks with tags (`n_track_tags`, 0.23 per tag, so five tags) have about 3.2 times more plays, holding the other features fixed; this is an association, not a cause, because tags exist mainly for already

popular tracks. Compared with the reference genre alternative, blues (-0.80) and country (-0.74) have lower play counts, electronic does not differ ($p = 0.48$). Title features have small effects, for example `name_length` (-0.006 per character, $p = 0.010$) and `name_equals_album` ($+0.20$, $p < 0.001$).

4.3 Diagnostics and unsuccessful results

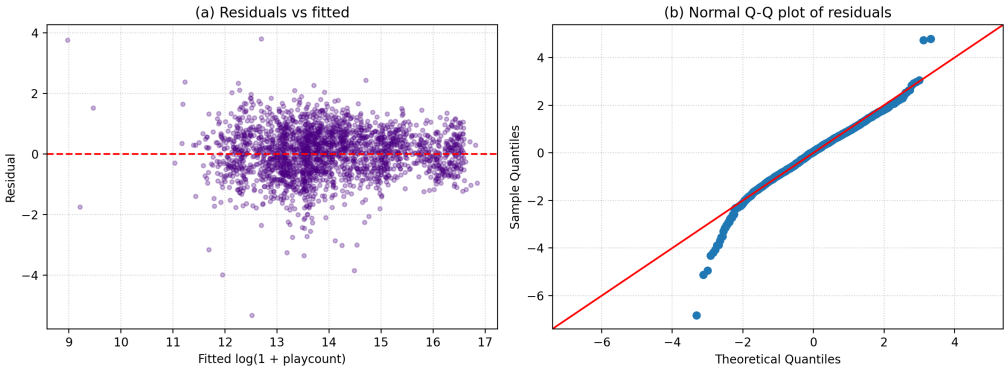


Figure 3: OLS residuals on the test set: residuals vs. fitted values (a) and normal Q-Q plot (b).

- **Heteroscedasticity:** the Breusch–Pagan test rejects constant variance (LM $= 1919$, $p < 0.001$).
- **Non-normal residuals:** left-skewed (skewness -0.47) with heavy tails (excess kurtosis 2.28), Figure 3(b).
- **Multicollinearity:** VIF is 70.6 for `log_wiki_length`, 68.7 for `has_wiki`, 23.1 for `log_artist_playcount` and 16.2 for `log_artist_listeners`; as a result `has_wiki` has a negative coefficient (-0.85) although its correlation with the target is positive (0.55). Without `has_wiki` and `log_artist_playcount` the largest VIF is 6.4 and the test R^2 is 0.711, so these two features were redundant.
- **Raw skewed predictors:** with raw `artist_listeners` and `artist_playcount` the condition number was $1.7 \cdot 10^{10}$ and both coefficients were negative despite positive correlations; logarithms fixed this.

5 Summary and Conclusions

Track popularity is mainly associated with the size of the artist's audience and with the availability of tags and descriptions; title features have only small effects. A linear model on the log target reaches a test R^2 of 0.71 on unseen artists, but constant variance and normal errors are not fully satisfied and some features are redundant. In the final version we will compare feature groups, test the possible leakage through `artist_playcount`, analyse outliers and use robust standard errors.

References

[1] Last.fm API documentation, `https://www.last.fm/api`. Accessed October 2026.

[2] Dataset (CSV, names replaced by identifiers): `https://drive.google.com/file/d/136-UX_rI5N2WoEPHlo5aL312i0R0QwI5/view?usp=sharing`.

[3] Pedregosa et al., Scikit-learn: Machine Learning in Python, JMLR 12, 2011.

[4] Seabold and Perktold, Statsmodels: Econometric and statistical modeling with Python, 2010.

[5] Last.fm API Terms of Service, `https://www.last.fm/api/tos`. Accessed October 2026.